

Lab Worksheet R03 — Bisection Search, Functions & Approximation Algorithms

Use: "Recitation-style Lab B" (40 min) of a 3-hour session. **Ground rule:** the TA never gives away answers — the TA only asks tracing questions.

Warm-up (about 5 min)

Original Problem 2: *a function to check whether a number falls in a given range*. Fill in the blanks, then predict the two printed lines before running: _____ / _____

```
def check_in_range(x, start, end):
    if x _____ start and x <= _____:      # (1) comparison (2) variable name
        return "Yes"
    else:
        _____ "No"                            # (3) keyword that sends a value back

print(check_in_range(3, 1, 5))
print(check_in_range(3, 5, 7))
```

Core exercises (about 30 min)

Exercise 1. Fourth root by bisection search (original Problem 1)

(a) **Worked example** — read the original solution, then run it with input 16.

```
x = float(input("Using bisection search calculate the forth root of: "))
epsilon = 0.01
low = 0
high = x
ans = (high + low) / 2
while abs(ans**4 - x) >= epsilon:
    if ans**4 > x:
        high = ans
    else:
        low = ans
    ans = (high + low) / 2
print("Forth root of " + str(x) + " is approximately " + str(ans))
```

(b) **One-line change** — change `epsilon = 0.01` to `epsilon = 0.0001`. Does the loop run more times or fewer? Why? _____

(c) **Fill in the blank** — complete the search step.

```

low = ____
high = ____
ans = (____ + ____) / 2
while abs(ans**4 - x) >= epsilon:
    if ans**4 > x:
        ____ = ans           # which bound moves here?
    else:
        low = ans

```

[TA tracing questions]

1. "For input 16, what are `low`, `high`, and `ans` right after the first pass of the loop?"
2. "Why is `abs()` needed in the loop condition — what could go wrong without it?"

Exercise 2. Perfect numbers (original Problem 3)

(a) **Worked example** — a perfect number equals the sum of its proper divisors.

```

def perfect_number(n):
    my_sum = 0
    for x in range(1, n):
        if n % x == 0:
            my_sum += x
    return my_sum == n

print(perfect_number(6))

```

(b) **One-line change** — change the test call to `perfect_number(28)`, then `perfect_number(50)`. Predict both outputs before running: ____ / ____

(c) **Fill in the blank** — complete the function body.

```

for x in range(1, ____):
    if n % x == ____:
        my_sum ____ x
return my_sum ____ n

```

[TA tracing questions]

1. "For `n = 6`, what is in `my_sum` after the loop finishes checking `x = 3`?"
2. "The function has no `print` — what kind of value does `return my_sum == n` send back?"

Exercise 3. Fourth root by incremental approximation (original Problem 4)

(a) **Worked example** — the key loop of the original solution (initial parameters: `epsilon = 0.01`, `ans = 0.0`, `increment = 0.001`, `num_guesses = 0`; negative input is rejected first).

```

while abs(number - (ans**4)) >= epsilon and ans**4 <= number:
    ans += increment
    num_guesses += 1

```

(b) **One-line change** — change `increment = 0.001` to `increment = 0.1`. What happens to `num_guesses`, and can the search now *miss* the answer? _____

(c) Fill in the blank — complete the loop.

```
while abs(number - (ans**4)) >= epsilon and ans**4 <= ____:  
    ans += ____  
    num_guesses += 1
```

[TA tracing questions]

1. "After 10 passes of the loop, what exactly is stored in `ans`?"
2. "Which of the two `and` conditions stops the loop when the guess overshoots the target?"

Self-check (about 5 min)

- ☐ I can trace `low`, `high`, `ans` through two passes of a bisection loop by hand.
- ☐ I can explain the difference between `return` and `print`, and what a Boolean return value is.
- ☐ I can explain the trade-off between a smaller `increment / epsilon` and the number of iterations.

Source

MIT OpenCourseWare, **6.100L Introduction to Computer Science and Programming Using Python** (Fall 2022), **Recitation 3** (`R03_problems.py` / `R03_solutions.py`). License: **CC BY-NC-SA 4.0**. Problems 1–4 of the original are used; only scaffolding was added. See <https://ocw.mit.edu> for Terms of Use.